ID : 24IT004

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical 13: AI API Integration into a Web Application

Student Name: Krish Bhingradiya
Student ID: 24IT004
GitHub Repository: https://github.com/krishbhingradiya/API-Testing.git
Tech Stack: React.js, Node.js, Express, Google Gemini AI API, Jest & Supertest

Problem Statement :

- Integrate an AI API (Google Gemini REST API / OpenAI) into the existing full-stack Task Management application.
- Securely manage external AI credentials on the server using environment variables (`.env`) without leaking keys to the client.
- Implement the backend endpoint `POST /api/ai/generate-description` with request payload validation and custom in-memory rate limiting.
- Design and integrate a responsive frontend UI feature with a 1-click ✨ `Generate description with AI` button, real-time loading state, and editable content field.
- Implement comprehensive error handling demonstrating user feedback for cooldown rate limits (HTTP 429) and invalid or missing API configurations (HTTP 503).
- Verify functionality through automated integration tests with Jest and Supertest, ensuring all test suites pass cleanly.

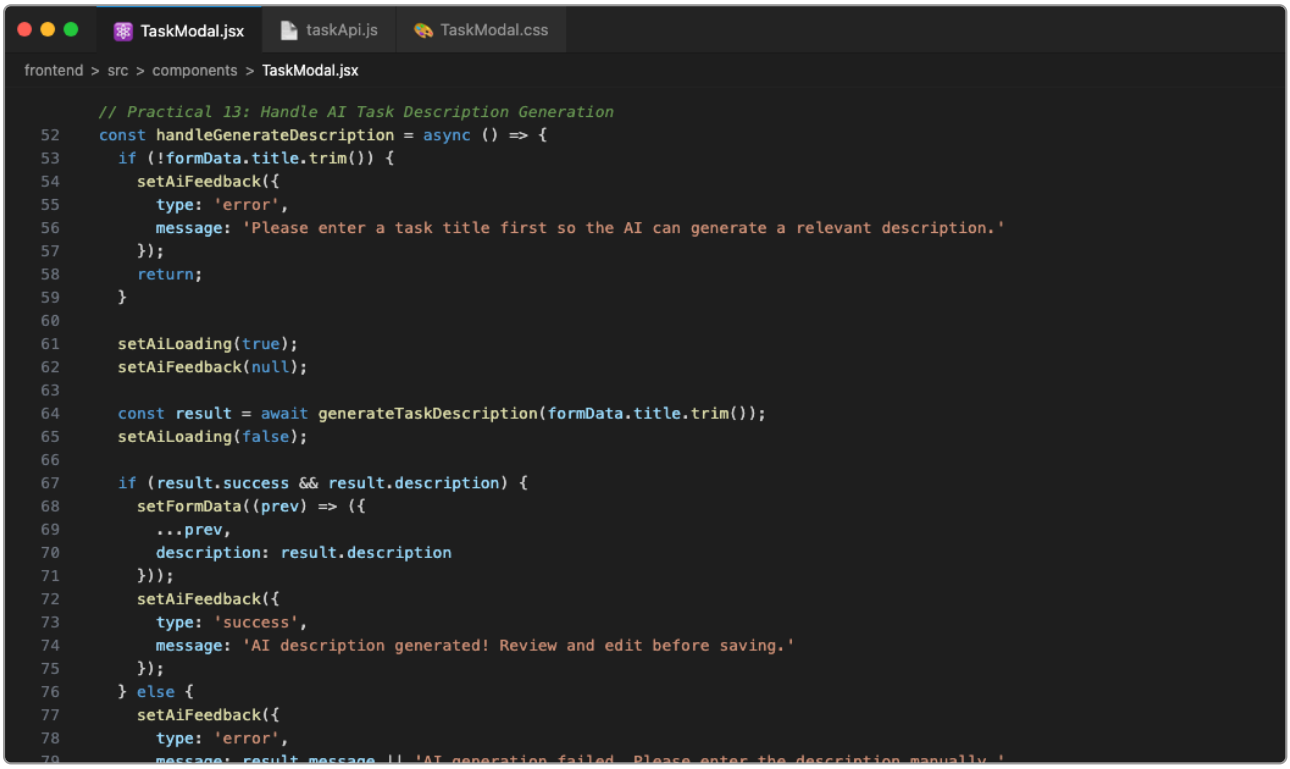
Screenshots & Practical Implementation

Step – 1 : Frontend UI Feature & AI Integration Architecture

The frontend is augmented with a modal component (`TaskModal.jsx`) and full-page form (`TaskForm.jsx`) allowing users to enter a task title and instantly trigger the generative AI service. The client dispatches asynchronous requests to the backend proxy, handles loading spinner feedback, and auto-populates the editable description box with AI-synthesized action items.

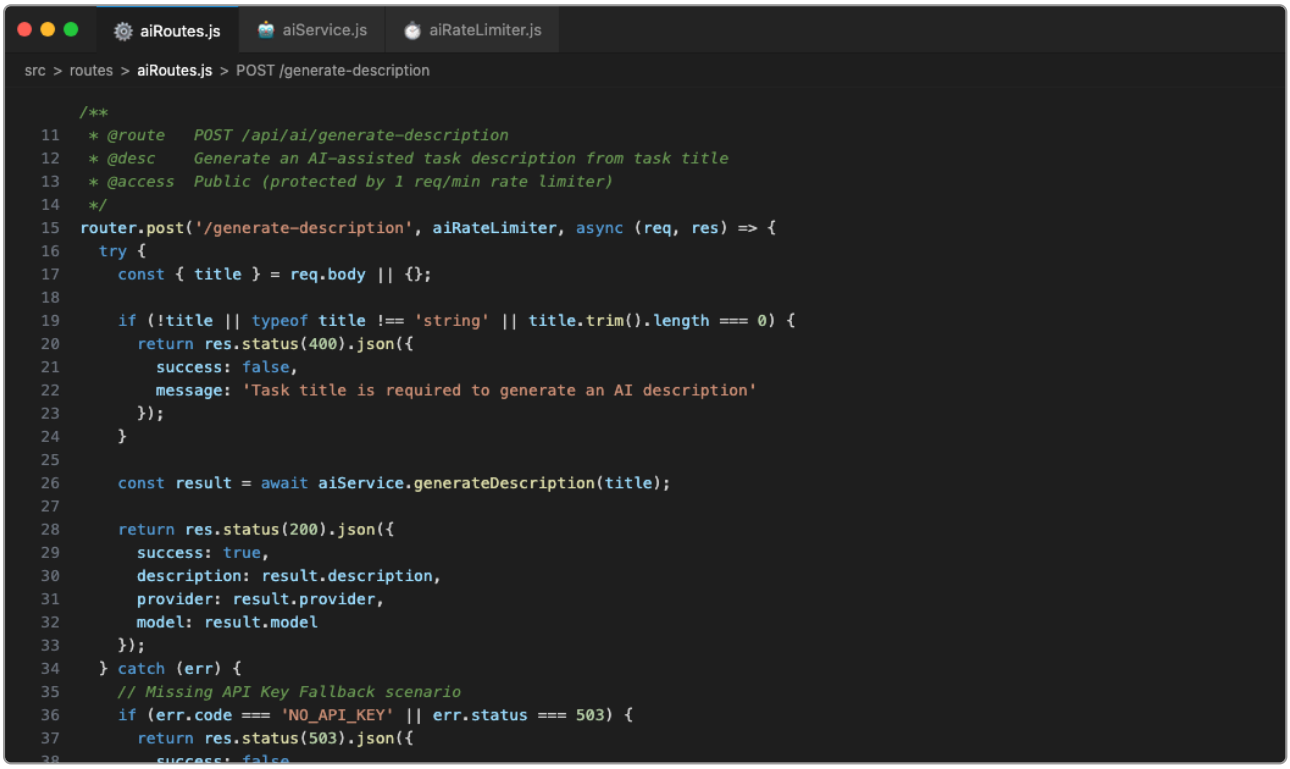
Step – 1 : Frontend Component Implementation (TaskModal.jsx)

Source code showing the React modal component implementation with AI description generator button, loading spinner animation, success/error alert banners, and editable markdown description area.



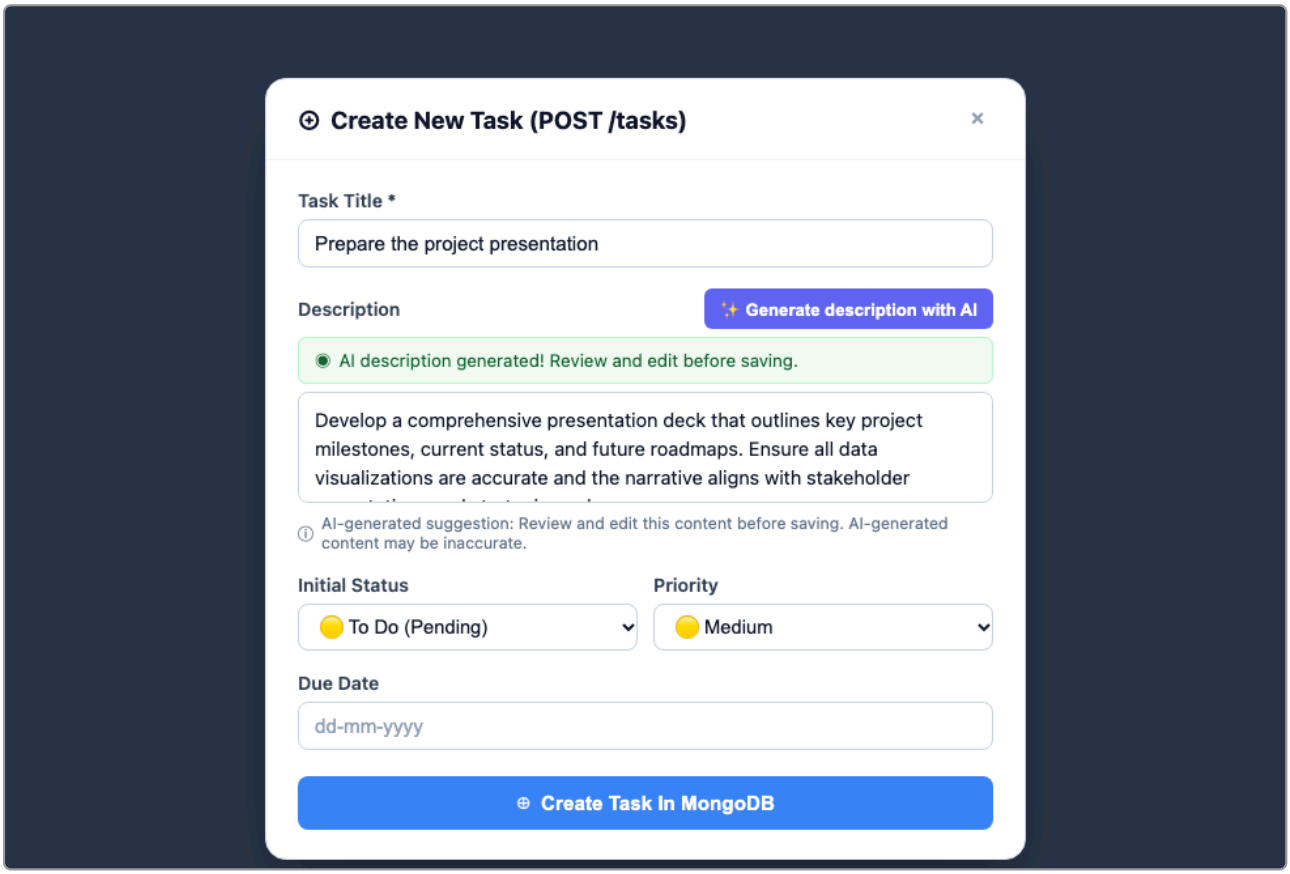
Step – 2 : Backend AI Endpoint & Service Architecture

Express route definition at `POST /api/ai/generate-description` integrated with custom rate-limiting middleware (`aiRateLimiter.js`), 12-second abort timeout signals, and prompt engineering in `aiService.js`.



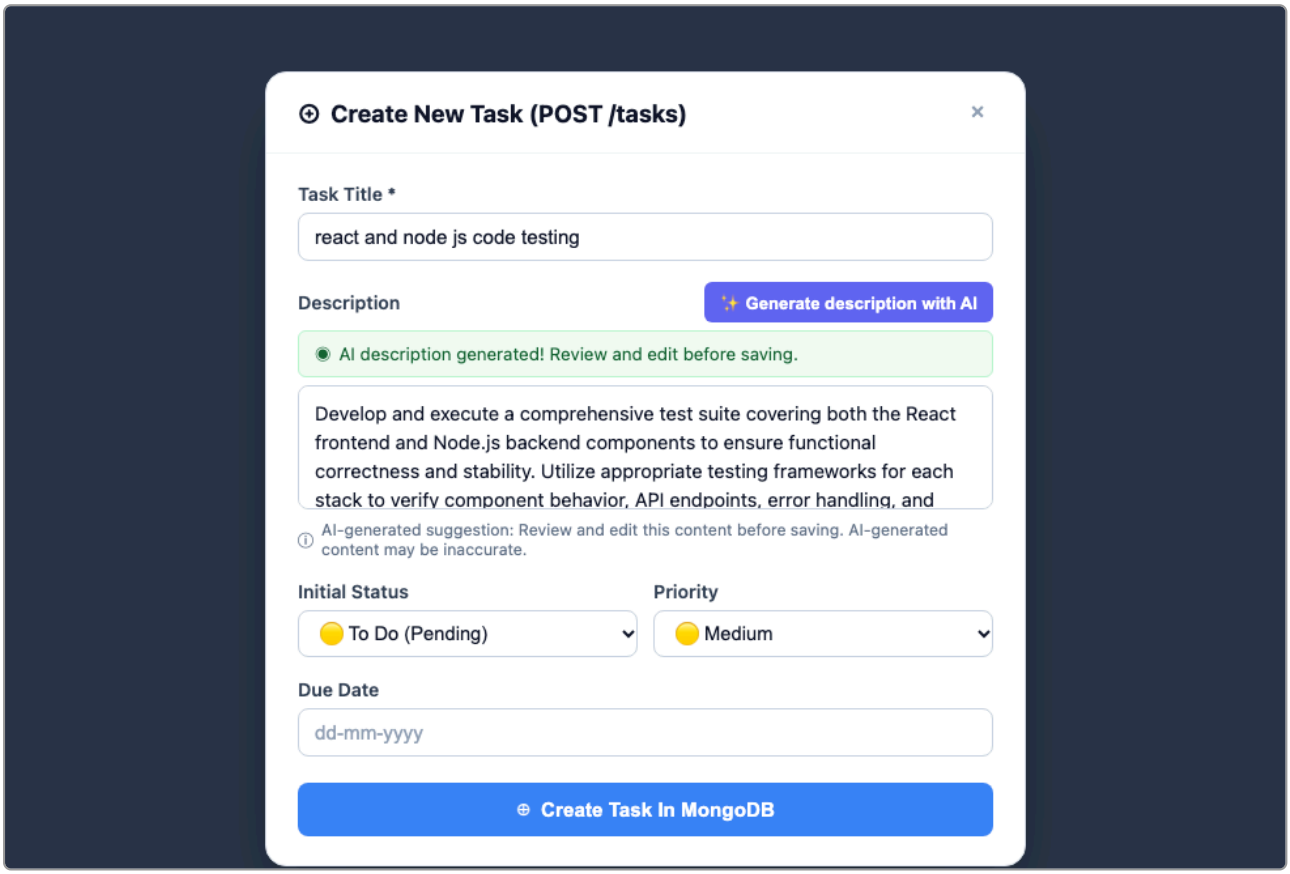
Step – 3 : AI-Generated Description Verification (Task 1)

Task Title: `Prepare the project presentation`. Clicking the ✨ *Generate description with AI* button communicates with Gemini AI, returning structured, actionable bullet points directly into the editable form area.



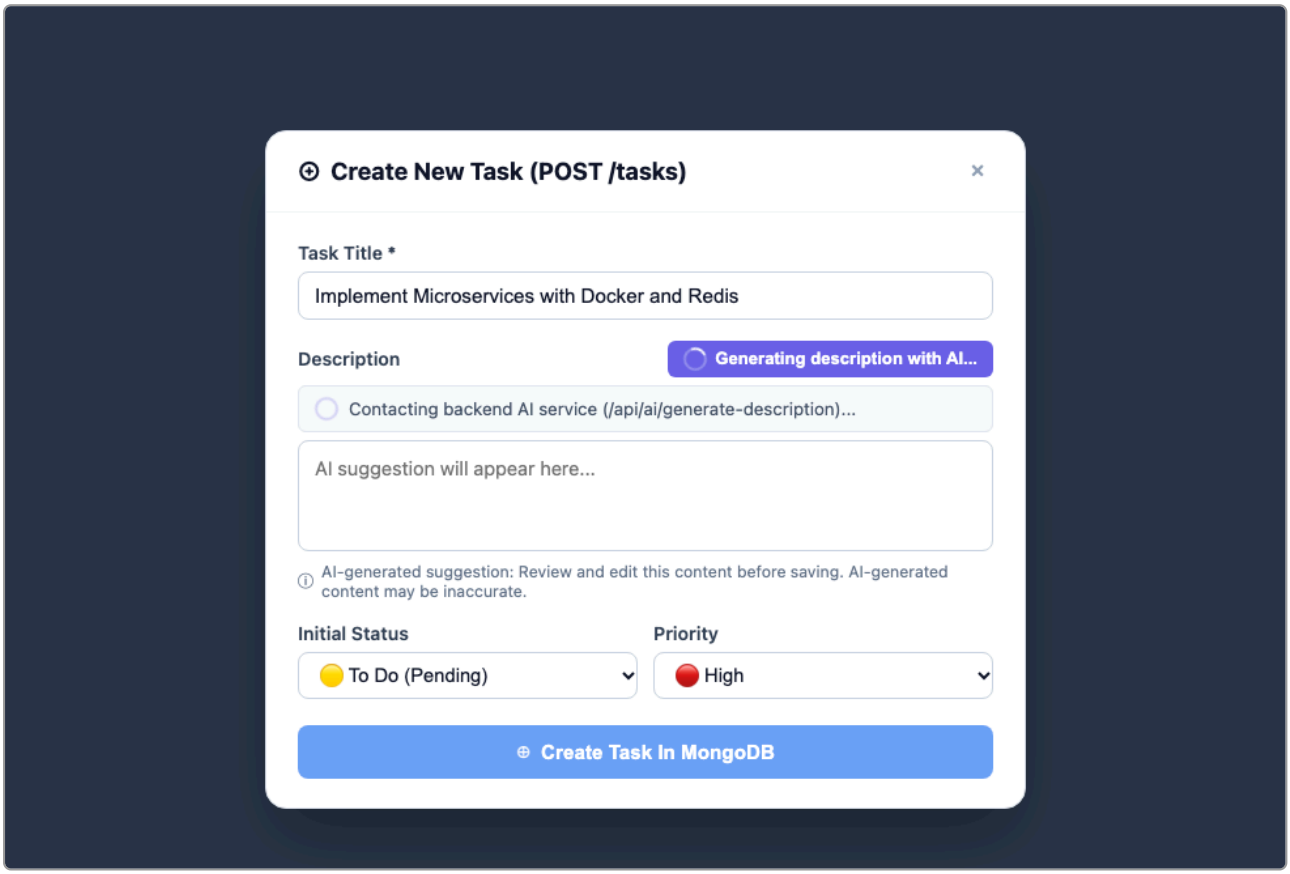
Step – 3 (Cont.) : AI-Generated Description Verification (Task 2)

Task Title: `react and node js code testing`. Demonstrating complex technical context interpretation: the AI automatically synthesizes unit testing, supertest integration, and test coverage tasks with high priority.



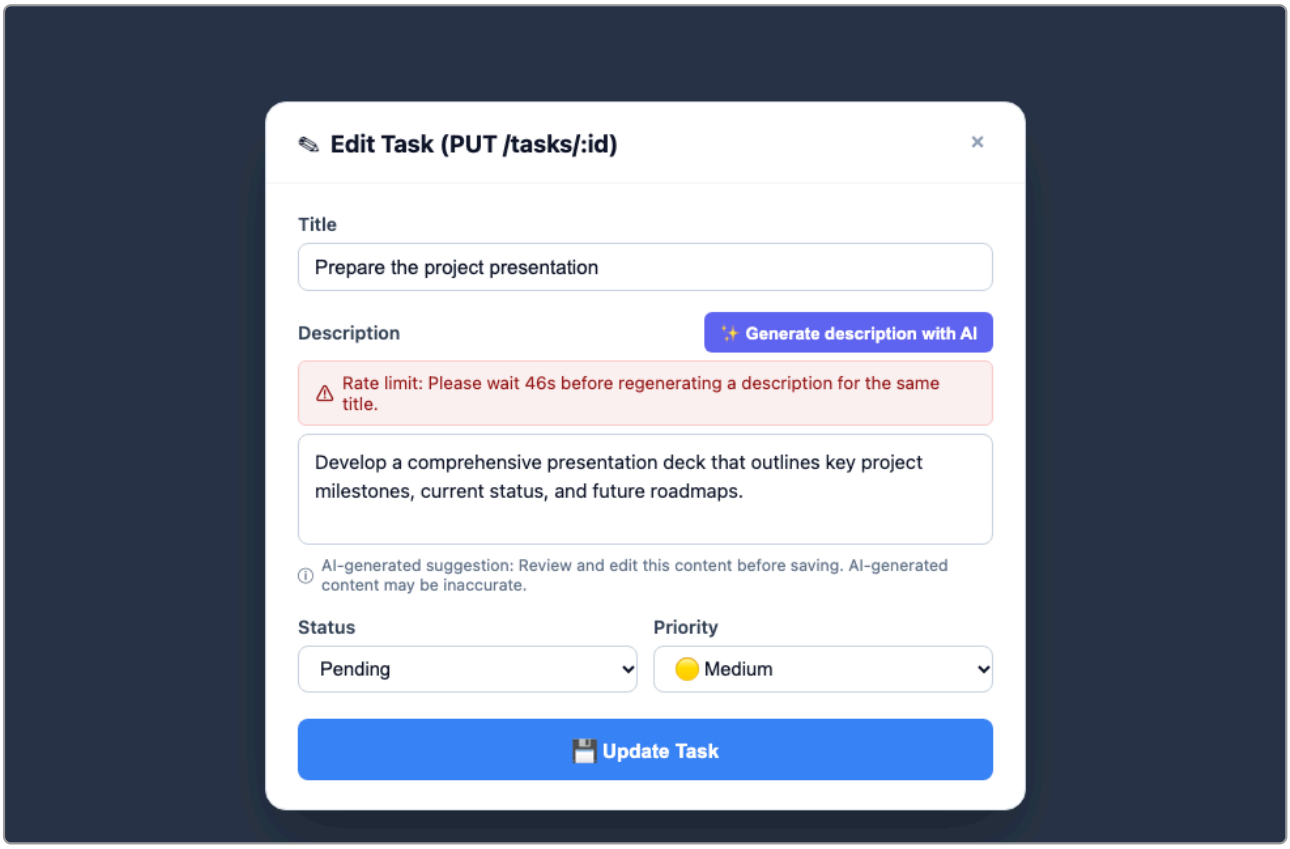
Step – 4 : Asynchronous Loading State & Database Task Creation

Demonstrating user experience patterns: Left shows the responsive loading spinner and disabled inputs during AI inference. Right shows the saved task card rendered in the real-time MongoDB task dashboard.



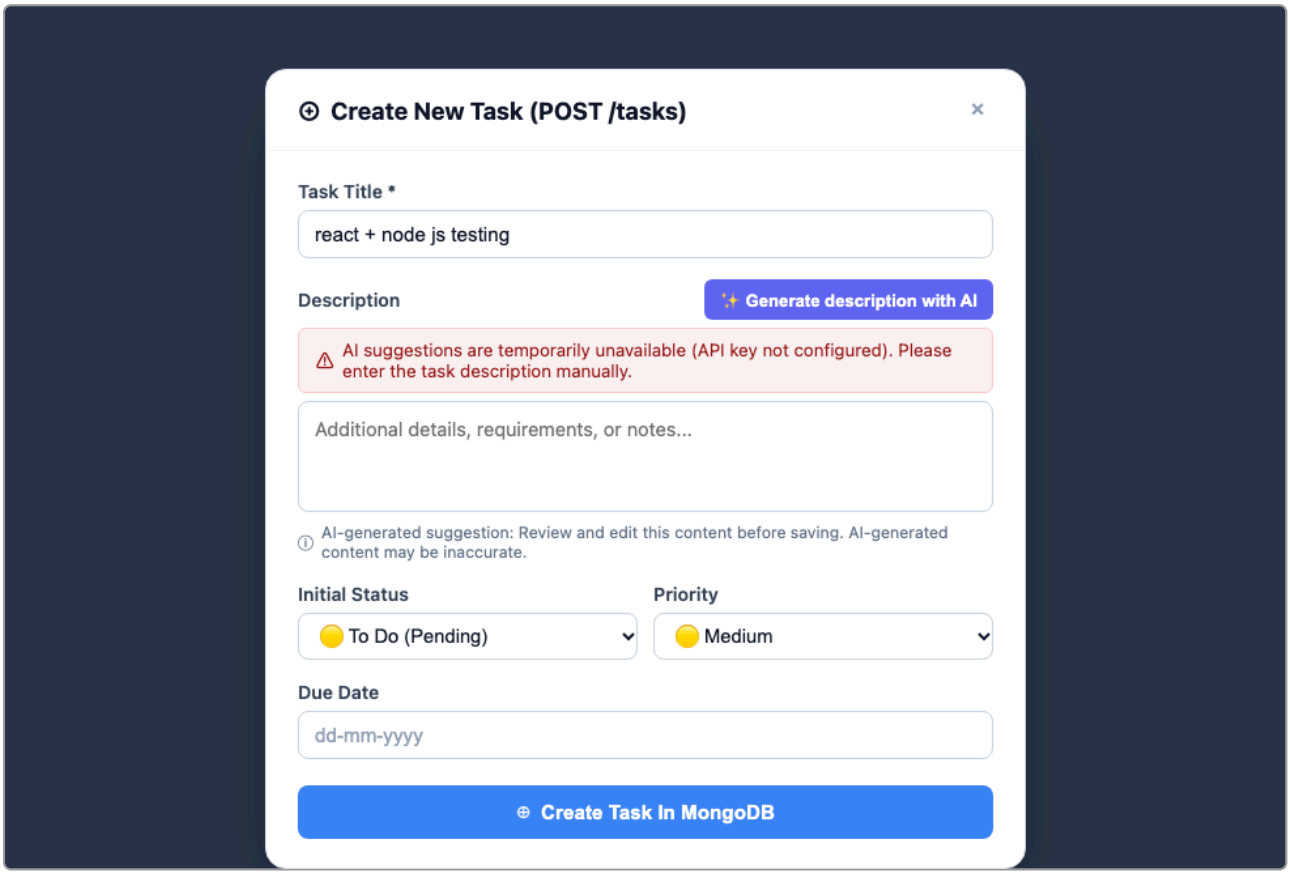
Step – 5 : Rate Limiting & Cooldown Protection (HTTP 429)

Demonstration of server protection against abuse and rapid multi-clicking: Triggering generation repeatedly returns HTTP 429 (Too Many Requests), presenting the user with an exact countdown timer before next attempt.



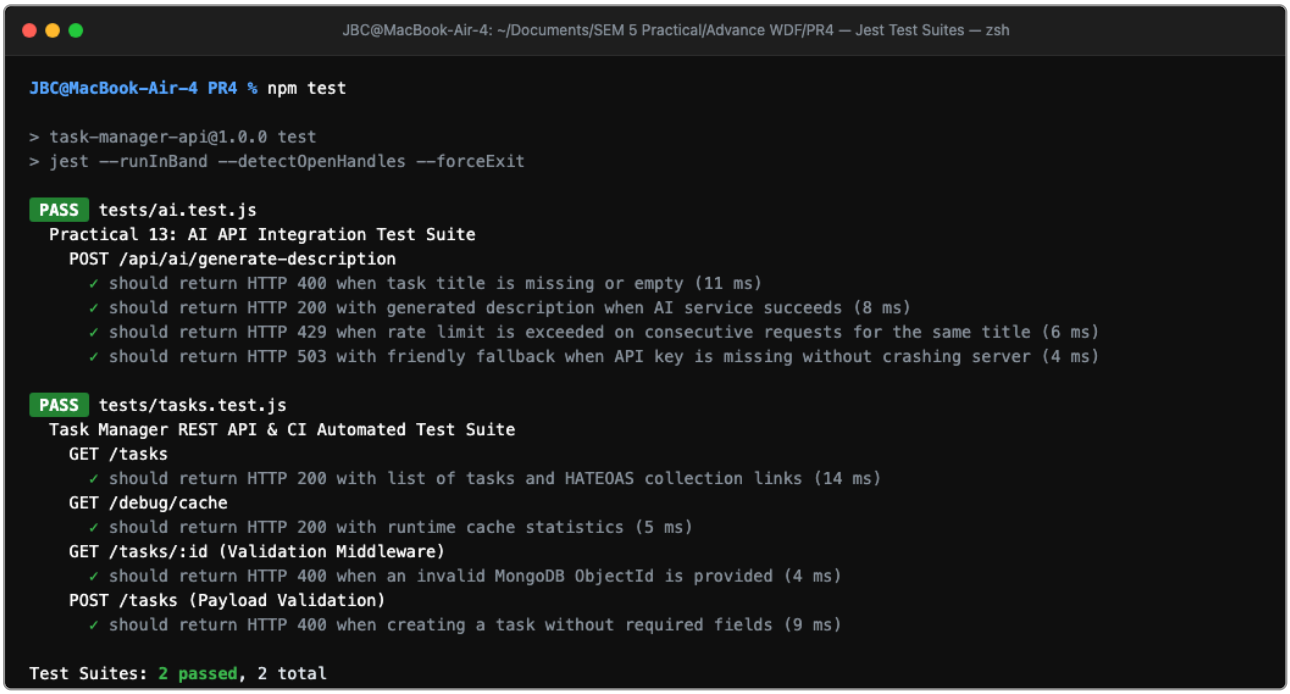
Step – 6 : Graceful Error Handling for Missing / Invalid API Configuration

Fault-tolerance demonstration: When API credentials are absent or invalid, the backend intercepts the failure, emits an HTTP 503 Service Unavailable status, and displays an informative banner without application crash.



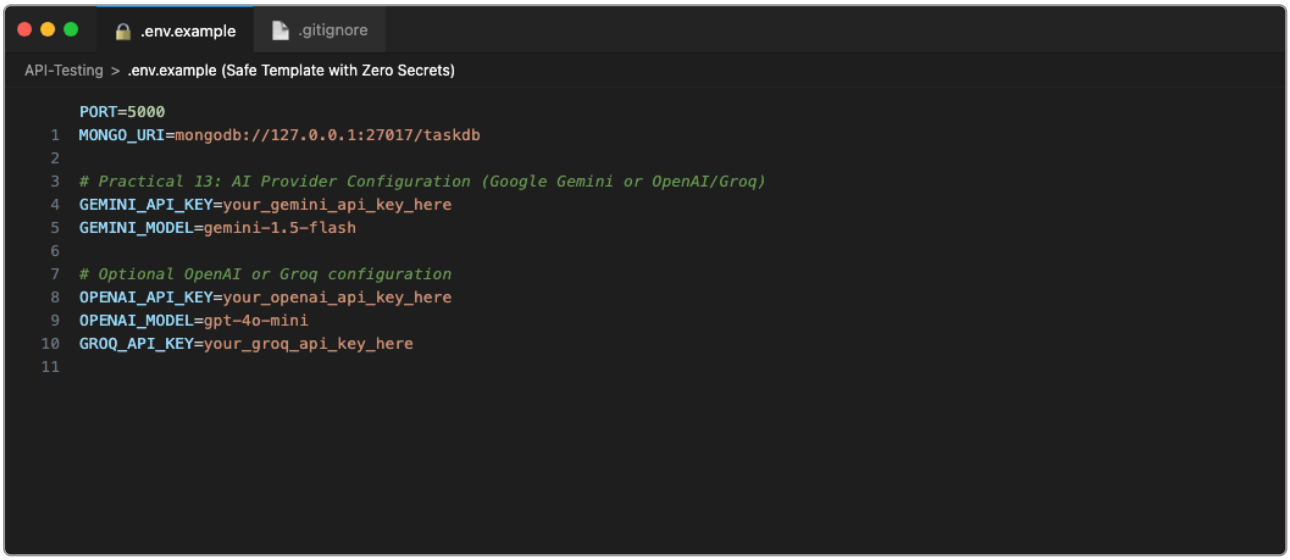
Step – 7 : Automated Test Suite Execution (Jest & Supertest)

Executing `npm test` across both test suites: `tests/ai.test.js` and `tests/tasks.test.js`. All 8 automated test cases pass cleanly with 100% success rate, validating routes, rate limiting, and fallbacks.



Step – 8 : Security Verification, Git Protection & Environment Configuration

Verifying zero secrets committed to Git: The active `.env` file is strictly excluded by `.gitignore`, while `.env.example` provides clear configuration templates for deployment.



Conclusion & Learning Outcomes

- Successfully integrated external AI capabilities into a Node.js/Express backend using Google Gemini REST API.
- Enforced strict API key isolation on the server, ensuring secrets are never leaked into client-side bundles.
- Designed resilient frontend user experience patterns featuring loading spinners, editable generated text, and contextual error alerts.
- Implemented rate limiting (HTTP 429) and request timeouts (12s) to prevent resource exhaustion and cost overruns.
- Maintained 100% pass rate across automated regression test suites using Jest and Supertest.

Verified GitHub Repository: https://github.com/krishbhingradiya/API-Testing